Week 6 Summary: Django Setup & Project Structure

Comprehensive guide covering environment setup, project vs. app architecture, core files, and request lifecycle.

1. Setup & Environment

- **Create Project Folder:** Initialize your workspace directory using terminal commands:

```
mkdir myproject && cd myproject
```

- **Create Virtual Environment:** Isolate project dependencies from the global Python installation:

```
python -m venv venv
```

- **Activate Virtual Environment:**
  - *Mac / Linux:* `source venv/bin/activate`
  - *Windows:* `venv\Scripts\activate`
- **Install Django:** Install the framework inside the active environment:

```
pip install django
```

- **Freeze Requirements:** Save installed packages to a dependency file:

```
pip freeze > requirements.txt
```

2. Project vs. App Architecture

- **Django Project (Main Container):** Created via `django-admin startproject project_name`. Holds global configurations, root settings, and global URLs.
- **Django App (Feature Module):** Created via `python manage.py startapp app_name`. A modular, reusable component built for a specific feature (e.g., users, products, blog).

> **Important Rule:** Don't forget to register your newly created app inside `INSTALLED_APPS` within `settings.py`!

3. Core Files & Their Roles

- **`manage.py`:** Administrative command-line utility used to interact with the project (e.g., running the server, executing migrations).
- **`settings.py`:** Central configuration file containing installed apps, middleware, database credentials, templates, and static file paths.
- **`urls.py`:** Maps incoming browser URL paths to their corresponding view functions.
- **`views.py`:** Handles application logic, processes requests using models and templates, and returns responses.
- **`wsgi.py / asgi.py`:** Entry points used to deploy the project to production web servers (synchronous and asynchronous respectively).

4. Request-Response Lifecycle

- **Start Server:** Run the development server using `python manage.py runserver`.
- **Request Flow Path:**
  1. Browser sends a URL request.
  2. Project `urls.py` routes the request to the correct app.
  3. App `urls.py` maps the specific URL to a view.
  4. `views.py` executes the view logic (interacting with models/templates).
  5. Returns an `HttpResponse` (rendered template or data response).
  6. Rendered output is displayed in the user's browser.

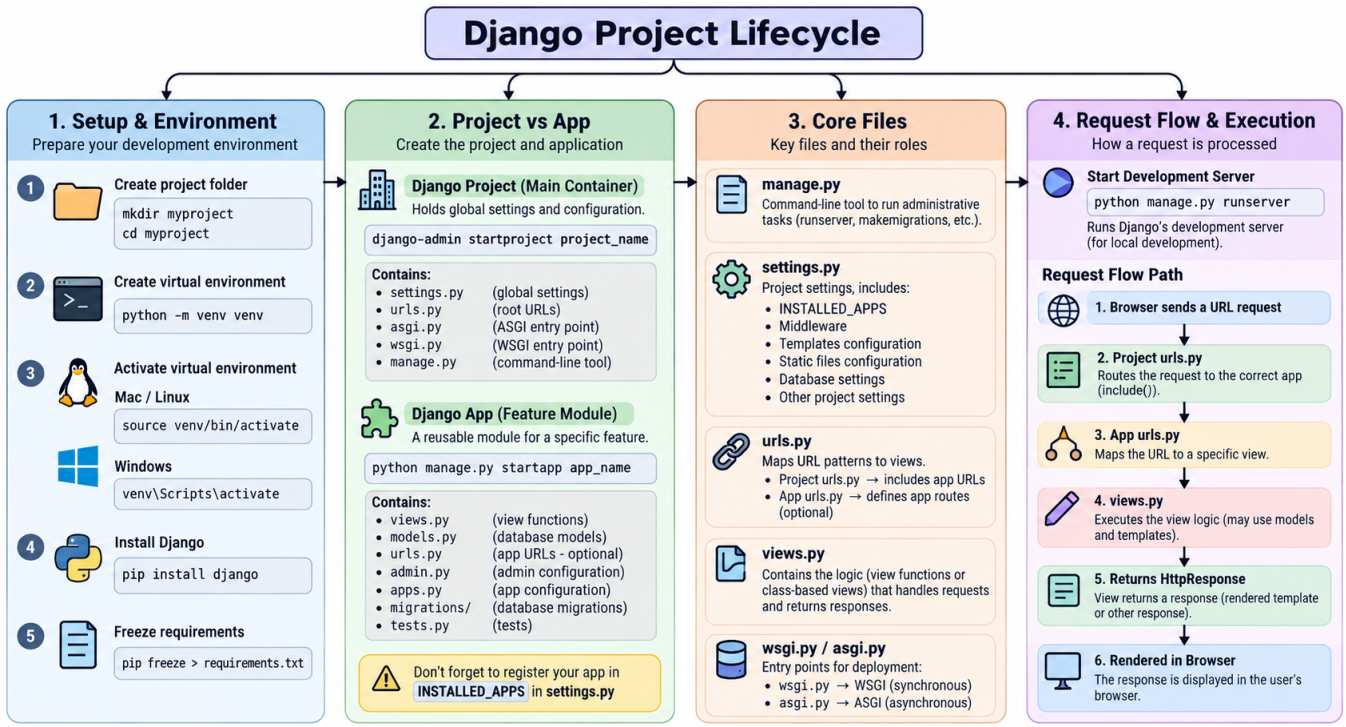


Figure: Complete Django Project Lifecycle & Structural Breakdown